

Entrega LAB3

1. Introducció

L'objectiu d'aquesta pràctica és implementar un servei de **multicast amb ordre total** utilitzant un algorisme distribuït. L'algorisme escollit és el que s'utilitza al sistema **ISIS**, el qual es basa en sol·licitar propostes de números de seqüència a tots els nodes d'un grup per acordar un ordre d'entrega comú.

El sistema està format per diversos *workers* (treballadors) que tenen un estat visual representat per un color. La necessitat de l'ordre total és crítica: si els missatges arriben en ordres diferents a diferents nodes, els colors finals divergeixen. El nostre objectiu és completar el mòdul `total.erl` perquè tots els workers mantinguin el mateix color independentment dels retards de xarxa (*Jitter*).

2. Implementació i Modificacions del Codi

L'únic fitxer que ha estat necessari modificar és **total.erl**. La resta de mòduls (`worker.erl`, `gui.erl`, `seq.erl`, `toty.erl`) s'han utilitzat tal com es van proporcionar.

A `total.erl`, s'han completat les funcions clau per gestionar les tres fases de l'algorisme ISIS:

1. **Enviament (request)**: Quan es vol fer un multicast, s'envia una petició (`request`) a tots els nodes (inclòs un mateix) per demanar un número de seqüència.
2. **Proposta (proposal)**: En rebre un *request*, cada node calcula el màxim entre el número acordat més alt (`MaxAgr`) i el proposat més alt (`MaxPrp`), l'incrementa i l'envia de tornada com a proposta. A més, el missatge es guarda a la cua de prioritat (`Queue`).
3. **Acord (agreed)**: L'emisor recull totes les propostes. Quan les té totes, tria la més alta i envia un missatge `agreed` a tothom. En rebre aquest missatge, els nodes actualitzen la cua, marquen el missatge com a lliurable i, si està al capdavant de la cua, el passen al *Master* (`deliver`).

El codi completat es troba al fitxer adjunt `total.erl`.

3. Testing

3.1. Preparació (Compilació)

Abans d'executar les proves, és necessari compilar tots els mòduls. Des de la consola d'Erlang:

```
None
c(seq).
c(gui).
c(basic).
c(worker).
c(total).
c(toty).
```

3.2. Aixecar els Nodes

El mòdul `toty.erl` s'encarrega d'aixecar automàticament els 4 nodes (P1, P2, P3, P4) i la interfície gràfica associada mitjançant la funció `init`. No cal aixecar-los manualment.

3.3. Executar la prova

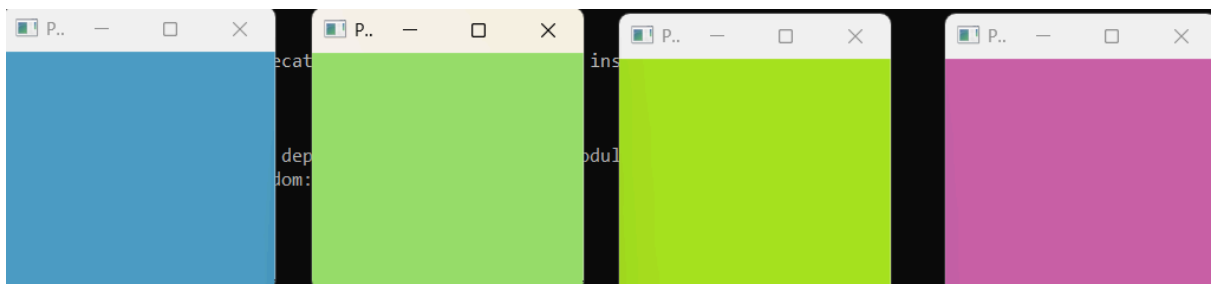
S'han realitzat dues proves principals per verificar el funcionament:

Prova A: Basic Multicast (Control) S'executa el sistema amb el protocol bàsic i un *Jitter* alt per simular retard de xarxa variable.

- **Comanda:** `toty:start(basic, 50, 100).`

```
7> toty:start(basic, 50, 200).
true
```

- **Resultat observat:** Els missatges arriben desordenats. Els colors de les finestres acaben sent diferents entre elles. Això confirma que sense ordre total, el sistema falla.

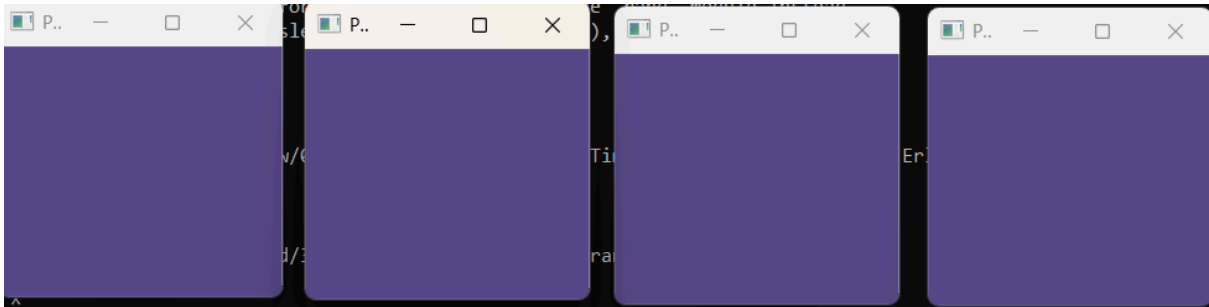


Prova B: Total Multicast (Solució) S'executa el sistema amb el protocol ISIS implementat.

- **Comanda:** `toty:start(total, 50, 100).`

```
9> toty:start(total, 50, 200).
true
```

- **Resultat observat:** Malgrat el *Jitter*, totes les finestres canvien de color alhora i mantenen exactament el mateix color (RGB) durant tota l'execució. Això valida la correctesa de la implementació.



4. Respostes a les Preguntes Obertes

A continuació es responen les preguntes plantejades a la pràctica.

Pregunta 1: Does it keep workers synchronized? Justify why.

Resposta: No, la implementació bàsica (basic) no manté els treballadors sincronitzats quan hi ha un factor de Jitter (retard aleatori) significatiu.

Això es deu al fet que el multicast bàsic envia missatges de forma asíncrona sense cap mecanisme de coordinació. A causa dels retards aleatoris introduïts pel paràmetre Jitter, un node A pot rebre el missatge 1 abans que el 2, mentre que un node B rep el 2 abans que l'112. Com que l'operació de canvi de color no és commutativa, l'ordre d'arribada altera el resultat final, provocant que els estats (colors) dels nodes divergeixin.

Pregunta 2: Total Order Multicast questions.

- **i) Does it keep workers synchronized?**

Resposta: Sí. Amb la implementació de l'algorisme d'ISIS (total), els treballadors es mantenen perfectament sincronitzats. L'algorisme garanteix que, encara que els missatges arribin físicament en moments diferents, no es "lliuren" (deliver) a l'aplicació fins que s'ha acordat un número de seqüència global únic per a tots13.

- **ii) Theoretical quantification of messages.**

Resposta: Per enviar un missatge de multicast a un grup de N nodes, el nombre total de missatges necessaris és 3N.

El desglossament és el següent:

1. **Fase de Request:** L'iniciador envia el missatge a tots els N nodes. (N missatges).
 2. **Fase de Proposal:** Cada node respon amb una proposta a l'iniciador. (N missatges).
 3. **Fase d'Agreement:** L'iniciador envia la decisió final (seqüència acordada) a tots els N nodes. (N missatges).
- Total: $N + N + N = 3N$.

- **iii) Compare with the basic multicast implementation.**

Resposta: El multicast bàsic té una complexitat de missatges de N (un enviament

directe a cada node), mentre que el d'ordre total en necessita $3N$.

Això implica que el cost de garantir la consistència (ordre total) és triplicar el tràfic de xarxa per cada missatge enviat. És un compromís necessari: perdem eficiència de xarxa i introduïm latència (esperar les respostes), però guanyem la garantia que tots els nodes processen la informació exactament en el mateix ordre.